

\$2.41 Trillion: The Cost of Poor Software Quality

- \$2.41 trillion. That is the total estimated cost of poor software quality — not globally over decades, but in the US alone in a single year (2022 CISQ report).
- \$1.52 trillion of that is directly attributable to technical debt — code that was written quickly rather than correctly, and now has to be maintained, worked around, or rewritten.
- That's equivalent to 6% of US GDP. To put that in perspective, it's comparable to the entire GDP of the UK.
- These aren't abstract numbers. They represent delayed projects, production outages, security breaches, and developers spending their days firefighting instead of building.
- The case for best practice isn't philosophical — it's financial.

Without Best Practice

- Here's a simplified example of what that looks like at the team level.
- Three features. Without best practice, Feature 1 costs £100k, Feature 2 costs £200k, Feature 3 costs £400k — totalling £700k.
- Why does the cost keep going up? Because each feature is built on top of an increasingly unstable, poorly understood codebase.
- The first team to cut corners "saved" time. Every subsequent team paid for it with interest.
- This is the compounding nature of technical debt — small shortcuts early on become expensive problems later.

With Best Practice (Cost Comparison)

- Now look at the same three features built with best practice: £110k, £110k, £110k — £330,000 total.
- Yes, Feature 1 costs a little more upfront. There's an investment in doing it properly.
- But Features 2 and 3 don't get harder — they stay consistent, because the foundation is solid and the codebase remains understandable.
- The saving isn't just £370,000. It's predictability. It's being able to estimate accurately. It's developers who aren't demoralised by constant firefighting.
- Best practice pays for itself — usually by the second feature.

Software Development in a Nutshell

- At its core, software development comes down to two things:
 1. **Build the right thing** — are we solving the actual problem? Do we understand what the user needs?
 2. **Build it right** — are we building it in a way that is maintainable, scalable, and understandable?
- Tonight I'm focused mainly on the second — building it right — because that's where best practice lives. But the two are connected: messy code makes it harder to pivot when you realise you built the wrong thing.

What is Best Practice?

- Definition: *"Techniques to build useful software in the most cost effective way possible."*

- Note the emphasis on *useful* — best practice isn't about writing elegant code for its own sake. It's always in service of delivering value.
- Also note *cost effective* — this isn't about perfection. It's about finding the approach that balances quality and pragmatism.
- Best practices are not rules handed down from on high. They are hard-won lessons from the industry, refined over decades, validated by experience. When you break one, there's usually a cost — you just might not see it straight away.

Naming

- Naming is one of the most underrated skills in software development. There's an old joke: "There are only two hard things in computer science — cache invalidation, naming things, and off-by-one errors."
- The name of a variable, function, or class is its documentation. If the name is wrong or vague, every developer who reads that code has to do extra mental work to figure out what it actually does.
- Good naming is free. It costs nothing at runtime and it pays dividends every time someone reads the code — including you, six months later.
- Ask yourself: if someone reads this name without any other context, will they understand what it does? If not, rename it.

Exercise: Find the Bug

- Provide three samples of code
- Explain how sample 1 is easy to understand
- Explain how sample 2 is the same as sample 1 but with bad naming
- Explain how sample 3 has the same bug but is impossible to understand

Best Practice and AI

- AI coding tools are genuinely impressive. GitHub Copilot, ChatGPT, Claude — they can generate a working function in seconds.
- AI amplifies your patterns — good and bad. If you practice good naming, clear structure, and small functions, the AI will produce better suggestions that fit your codebase.
- More importantly: AI can write code faster than ever, but you still need to *read* it, *review* it, and *own* it. The bar for code literacy hasn't gone down — if anything, it's gone up.
- Best practice matters even more in an AI-assisted world, because the volume of code being produced is higher.